

Research completed in 6m ·

012345678901234567890123456789 citations ·

012345678901234567890123456789 searches

SPI Controller Microarchitecture Specification –
Master/Slave

SPI Controller Microarchitecture Specification – Master/Slave

Executive Summary: This document presents a comprehensive microarchitecture specification for a Serial Peripheral Interface (SPI) controller, intended as a golden reference for RTL designers and DV engineers. It covers the full design-from first principles through RTL implementation details. Key elements include a block-level diagram, module responsibilities, APB bus interface, [SPI signal definitions \(MOSI, MISO, SCLK, SS_n\)](#), clocking and CDC strategy, configurable clock divider, TX/RX FIFOs, master/

slave FSMs, register map, timing diagrams for all four SPI modes, bit/byte transfer examples, error handling (underrun/overflow, mode fault), interrupts and optional DMA support, power/reset behavior, test/verification hooks (monitors, scoreboards), SVA assertions, functional coverage points, formal property suggestions, performance analysis, debug features (trace, counters), and implementation/integration notes. Unspecified parameters (e.g. bus type, FIFO depth, data width) are clearly marked with recommended defaults and rationale.

Throughout, explanations are given at three levels of abstraction (“Beginner/Intermediate/Senior”) for key design decisions, emphasizing why decisions are made (DV concerns and real silicon use) as well as what is done. Citations are provided from primary references (datasheets, IP docs, standards, etc.) for factual details. A concise checklist is included for

designer handoff to DV.

1. Overview and Requirements

The SPI controller is a synchronous serial master/slave peripheral interface that connects a host bus (APB, AHB, or AXI) to one or more SPI devices (slaves). It supports full-duplex data transfer using four signals (SCLK, MOSI, MISO, and one or more SS_n lines). By convention:

Master (controller) drives the clock (SCLK) and chip-select (SS_n) lines.

Slave (target device) responds on MISO when selected.

MOSI carries data out of master into slave;

MISO carries data out of slave into master.

Typical word-sizes are 8 bits, 16 bits, or 32

bits; this IP supports a configurable data width (default 8).

Four SPI modes (0–3) are supported, defined by Clock Polarity (CPOL) and Clock Phase (CPHA) settings (see Section 5).

Design Goals and Features (DV-driven): The controller must implement all four SPI modes, full-duplex transfers, multi-slave support, configurable data width and frame length, and robust error/interrupt handling. It includes TX and RX FIFOs to decouple bus and SPI timing, a programmable clock divider for SCLK, configurable CPOL/CPHA, and optional DMA handshaking. DV requirements include: verifying correct timing (setup/hold, CPOL/CPHA behavior), exercising all modes and data lengths, handling FIFO underrun/overflow, testing SS behavior, and checking interrupt generation and error flags. These requirements form the basis

of the verification plan and coverage model.

Protocol Context: SPI is a *de facto* standard (no single ratified spec). It uses a simple master/slave paradigm: the master pulses the clock and asserts a slave's SS line low to enable a transaction. Data is shifted in/out one bit per clock edge in both directions (full-duplex). A typical transfer consists of N bit-clocks during which the master and slave each shift out one frame of data into each other's shift register.

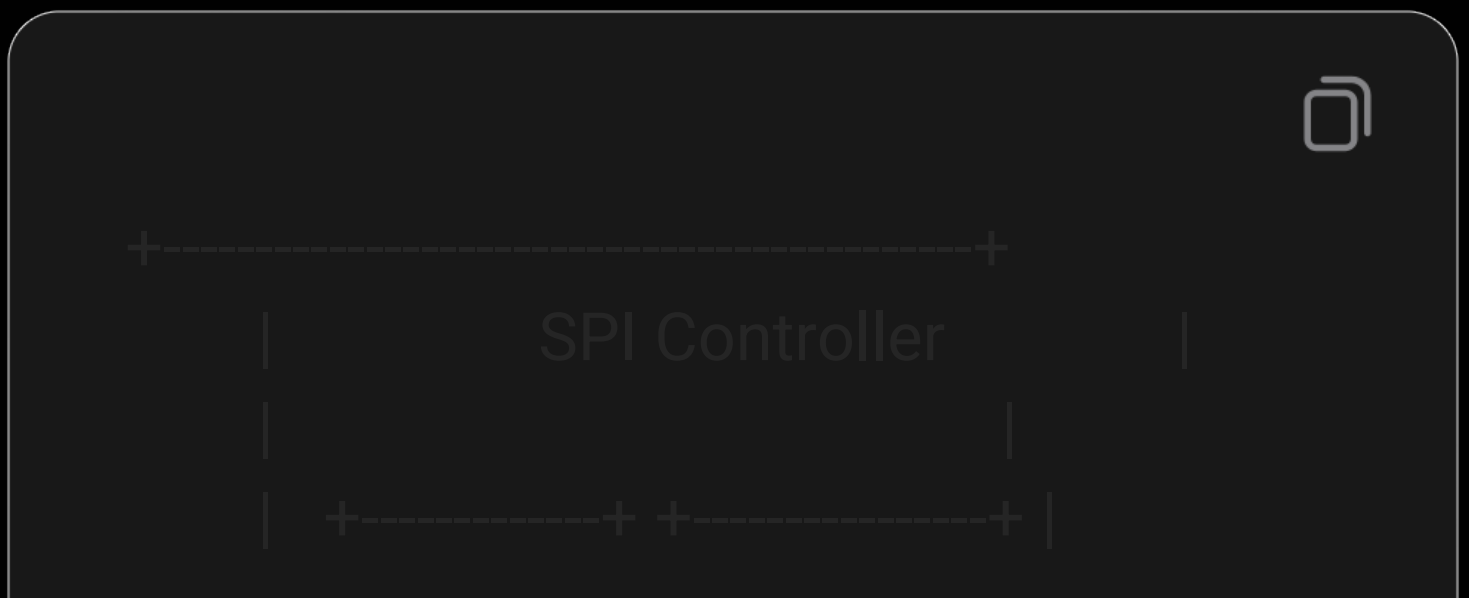
Popular applications include memory and sensor interfaces, where the SPI controller may interface to flash/EEPROM or ADC/DAC devices.

Non-goals: This spec focuses on a bare-metal SPI controller (with optional DMA). It does *not* cover higher-level flash protocols (XIP commands, OTP commands, etc) beyond basic data transfer. It assumes an APB-like bus interface (for register access); support for AXI/AHB can be added similarly.

2. Top-Level Architecture

The top-level SPI controller block (Figure 1) is subdivided into the following major modules: an APB bus interface, a set of Control/Status registers, a Clock Divider, TX FIFO and RX FIFO buffers, an SPI Protocol Engine (FSM for master/slave shifting), and an Interrupt Controller. The APB interface exposes memory-mapped registers for CPU control and status. The core logic implements the SPI protocol and data flow.

luaCopy



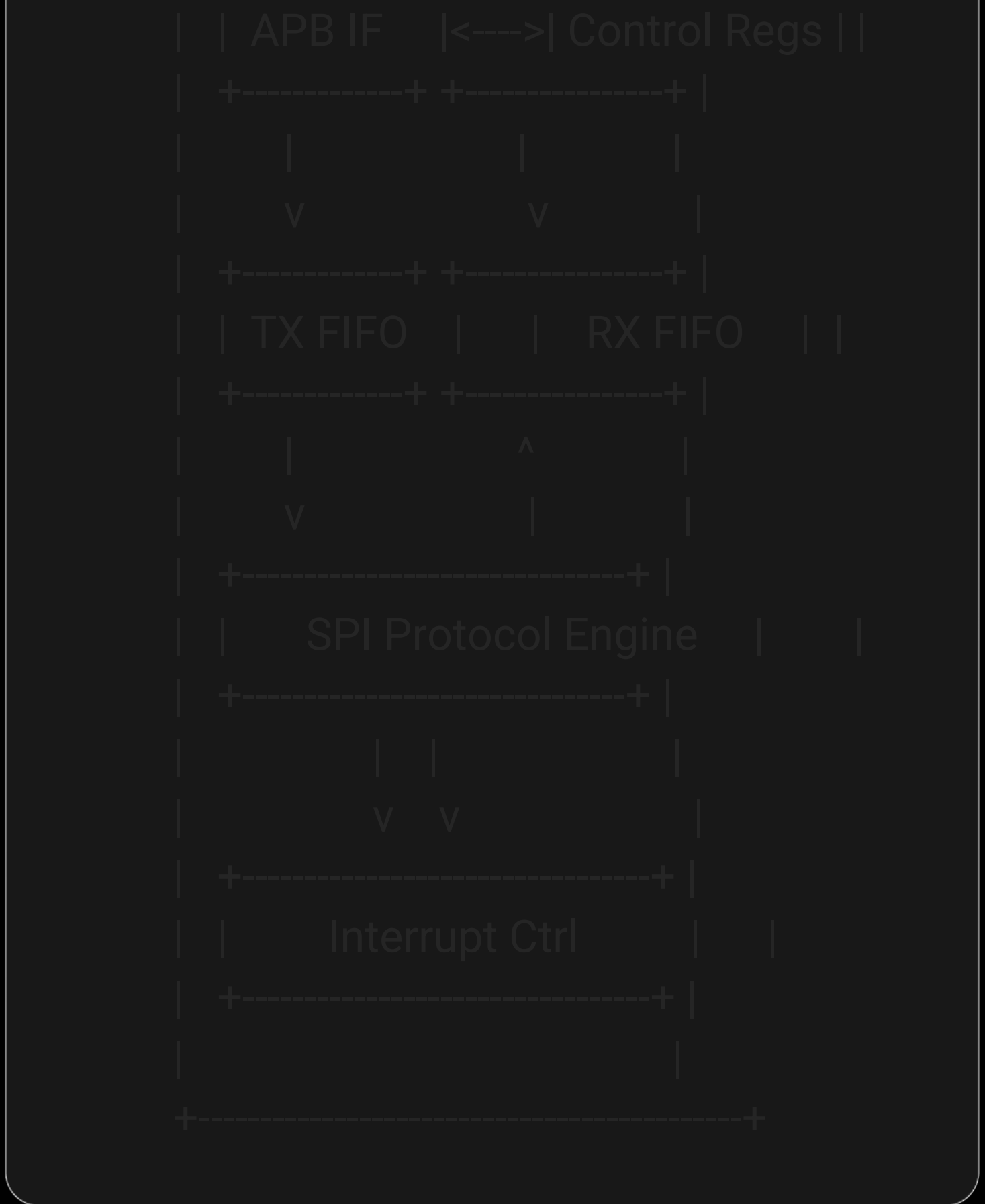


Figure 1. Top-level SPI Controller block diagram.

Each submodule has clear responsibilities:

APB Interface – Implements AMBA APB3/4

slave protocol. Decodes psel, penable, paddr, etc., handles reads/writes to internal registers, and drives pready and pslverr.

[Designed to follow ARM APB spec for timing and handshake.]

Control/Status Registers – A bank of CSR registers (CTRL, STATUS, BAUD, etc.) controlling the SPI engine. These are memory-mapped via APB and hold configuration bits (enable, CPOL/CPHA, frame length, interrupt enables, etc.) and status flags (busy, TX/RX FIFO level, errors).

Clock Divider – Generates the SPI serial clock (SCLK) from the system/APB clock (pclk_i). It holds a programmable divisor (BAUD register). When SPI is active, it toggles an internal clock enable at the divided rate.

TX FIFO – A FIFO buffer (default depth 16, configurable 4–64) for outgoing data. The

CPU writes data into TXDATA register, which pushes into TX FIFO. When the SPI engine is ready, it pops data from TX FIFO bit-by-bit to MOSI.

RX FIFO – A FIFO buffer (same default depth) for incoming data. The SPI engine shifts in bits from MISO into a shift register; when a complete frame is received, it pushes it into RX FIFO. CPU reads from RXDATA register to pop RX FIFO.

SPI Protocol Engine – The central FSM engine. In *master mode*, it asserts the selected SS_n, toggles SCLK (according to CPOL/CPHA), and shifts bits through MOSI/MISO; in *slave mode*, it waits for SS_n and SCLK edges to shift data. Handles full-duplex shifting and completion of frames.

Interrupt Controller – Monitors status flags (TX empty, RX full, transfer done, mode fault, underrun/overflow) and, if enabled, raises an

interrupt (spi_irq_o) for the CPU. Supports configuration for which events generate interrupts.

These modules interact as follows: APB writes to CTRL to start a transfer (e.g. set enable and SS_n). The TX path is: CPU → TXDATA register → TX FIFO → SPI engine → MOSI. The RX path is: MISO → SPI engine → RX FIFO → RXDATA register → CPU. Control path: APB writes configure the clock divider, SPI mode, enable bits. Interrupt path: status bits feed the interrupt logic, which outputs spi_irq_o.

Key Concepts (Why/What): Bits are transmitted via shift registers inside the SPI Engine.

Internally, both master and slave use shift registers (each acting like a circular buffer) so that every SCLK edge simultaneously shifts out one bit to the other device and shifts in a bit from the other device. A detailed state machine

controls these shift registers based on CPOL/CPHA and the SS_n signal. The FIFOs decouple the relatively slow APB domain from the faster SPI domain, allowing back-to-back transfers without CPU stalls.

3. Interfaces

[3.1](#) APB Bus Interface (Configuration and Status)

The SPI controller presents a standard AMBA APB slave interface for CPU access. Ports (from the block's perspective) include:

Clock/Reset: pclk_i (system clock), presetn_i (active-low reset).

APB signals: psel_i, penable_i, pwrite_i, paddr_i, pwn_data_i, prdata_o, pready_o,

pslverr_o.

Beginner: The APB interface simply allows the CPU to read/write registers.

Intermediate: The RTL implements a finite-state machine to comply with APB3 timing (IDLE → SETUP → ACCESS). It must sample psel/pwrite in SETUP, drive pready in ACCESS, and allow back-to-back transfers.

Senior: Use a small FSM or logic slice to pipeline reads/writes; ensure pready_o defaults high to avoid deadlock, and set pslverr_o only on invalid addresses if implemented.

The APB register map (Section 6) defines which CSR is accessed by each paddr_i. For example, a write to address 0x00 might set the CTRL register. The APB logic demultiplexes read/write to the appropriate CSR. It should fully comply

with the AMBA APB specification.

3.2 SPI I/O Interface

The SPI controller has the following physical I/O (external SPI pins):

spi_clk_o (output): SPI serial clock (SCLK), driven by master mode; tri-stated or held idle in slave mode.

spi_mosi_o (output): Master Out Slave In data (from controller to slave).

spi_miso_i (input): Master In Slave Out data (from slave to controller).

spi_csn_o (output, vector or single): Chip-select lines (active low) for slaves. Can be a 1-bit bus or N-bit multi-slave select (use default 1 and show optional multi-slave).

spi_irq_o (output): Interrupt request line, active high when an enabled event occurs.

Each pin behaves as follows:

SCLK: Idle level is determined by CPOL (0 or 1). During a transfer, SCLK pulses at the rate set by the clock divider. For master mode, the clock is toggled internally. In slave mode, SCLK is an input to the SPI engine (so MISO/MOSI shift on observed edges).

SS_n (CS): Active low. The master asserts (drives low) the selected slave's SS_n before sending clocks. On deassert (high), the transfer ends. The slave logic latches SS_n transitions to start/stop shifting. In multi-master systems, mode-fault logic (see Error Handling) monitors SS_n as input to detect collisions.

MOSI/MISO: Data lines operate as full-duplex shift lines. MOSI is driven by master (or internal loopback); MISO is driven by slave (internal or external). When no slave is selected, MISO should be tri-stated by slaves

(high-Z) so they don't conflict. Our design assumes proper tri-state on external devices.

A recommended interface table (APB and SPI) is shown in [22]. For brevity, Table 1 below shows the essential ports:

Bus Interface (APB)	Direction	Description
pclk_i	in	APB clock
presetn_i	in	APB reset (active low)
psel_i	in	APB select
penable_i	in	APB enable
pwrite_i	in	APB write enable
paddr_i	in [N:0]	APB address

		bus
pwdata_i	in [31:0]	APB write data bus
prdata_o	out [31:0]	APB read data bus
pready_o	out	APB ready/ idle
pslverr_o	out	APB error flag
SPI I/O		
spi_clk_o	out	SPI serial clock
spi_mosi_o	out	Master Out (to slave)
spi_miso_i	in	Master In (from slave)

spi_csn_o	out [SS_CNT-1:0]	Chip-select (active low)
spi_irq_o	out	Interrupt output

Table 1. Top-level interface signals.

3.3 Clocks and CDC

The controller may operate with multiple clock domains. At minimum, the APB bus is one clock (pclk_i domain) and the SPI serial clock is another domain internally generated (though it could also be gated from pclk_i).

Clock Domains: The main clock domain is pclk_i. The SPI engine uses a derived clock for SCLK, typically a divided version of pclk_i. In master mode, SCLK is generated by a clock divider (see next section). In slave

mode, SCLK is an external clock, but the SPI engine still synchronizes to it.

Clock Divider: The BAUD register holds a divisor. For example, if pclk_i is 100 MHz and BAUD=9, the SPI clock toggles every 10 cycles (generating 5 MHz SCLK). The divider logic is a simple counter that toggles a flip-flop at the end of the countdown. *Beginner:* It's just a counter that toggles a bit. *Senior:* Consider rounding and jitter—e.g. do not rely on the last partial cycle; ensure the counter is reset properly at end-of-transfer to avoid clock glitches.

CDC Strategy: Since MOSI, MISO, and SS_n lines toggle in the SPI clock domain, whereas CPU/CSR logic is in the APB domain, signals crossing domains must be synchronized. Key crossings:

Master Mode: The division result (SCLK) can directly drive SPI pins (no crossing

needed). Data bits originate from APB (TX FIFO) clock domain; before shifting, TX data words are loaded into a shift register on SCLK domain at the moment SS_n is asserted or the first clock edge. This load should be done with proper synchronization (e.g. use a handshake or dual-flop synchronizer if pclk_i and SPI clock are asynchronous).

Slave Mode: MISO data driven by SPI clock domain travels to APB domain (into RX FIFO). It must be sampled by a flip-flop in the APB clock domain to avoid metastability. Typically, implement a two-flop synchronizer on the MISO->RX_FIFO push signal or use an asynchronous FIFO with handshake. SS_n and SCLK (external) also cross into APB logic (for example, to set STATUS or generate interrupts); these should be sampled by synchronizers.

Reset and CDC: If asynchronous reset is used, ensure all synchronizer flops are reset to a known state.

CDC Note: OpenTitan's SPI Device requires the core clock to be at least 1/4 the SPI clock to meet CDC constraints, which is a good design guideline: faster core clock relative to SCLK makes safe synchronization easier.

In practice, common CDC practice is used: double-flop synchronizers for single-bit signals, or use Gray-coded handshakes for multi-bit counters, ensure FIFO crossing (if any) is safe. Plan review of all net crosses.

4. Core Modules

4.1 Control and Registers

A set of 32-bit registers holds configuration and status. The key registers (at default addresses) are:

Address	Name	Access	Description
0x00	CTRL	R/W	Control: Enables, master/ slave select, CPOL/ CPHA, LSB/ MSB, start transfer
0x04	STAT	R	Status:

			Busy flag, TX FIFO level, RX FIFO level, error flags
0x08	TXDATA	W	TX data input (pushes into TX FIFO)
0x0C	RXDATA	R	RX data output (pops from RX FIFO)

0x10	BAUD	R/W	Clock divisor for SCLK
0x14	FIFO	R/W	FIFO config (enable FIFO, threshold s, wordlen)
0x18	INT_EN	R/W	Interrupt enables for events (TX empty, RX full, mode fault,

			etc.)
0x1C	INT_ST	R/W1C	Interrupt status (read clears flags)
0x20	MODE	R/W	SPI mode (slave/master, CPOL, CPHA)
...	...	...	Reserved / extended features

Table 2. Example register map (configurable

addresses).

These registers are accessed via APB. For example, writing to CTRL with the “start” bit (and SS_n values) initiates a transfer. The STAT register provides read-only flags (e.g., BUSY, underflow). TXDATA/RXDATA are pseudo-registers that interface to the FIFOs. Specific bit-fields (enable, mode, FIFO enable, etc.) should be defined (as per project conventions).

Design Perspective: Each register corresponds to a flip-flop bank with defined bit-fields.

Designers should ensure registers reset to known defaults. DV engineers will verify register R/W behavior, proper field widths, and that reserved bits read as zero.

4.2 Clock Divider

The clock divider is a simple integer down-counter. It divides pclk_i to produce SPI SCLK.

Key points:

The divider value is loaded from the BAUD register. If $\text{BAUD} = N$, SCLK toggles every $(N+1)$ pclk_i cycles (for a $(N+1):1$ division ratio).

The divider should be a repeating counter: e.g., if $\text{BAUD} = 3$, it counts $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ then toggles SCLK, then repeats.

Reset Behavior: At reset, SCLK idle state must be consistent with CPOL setting, so you might preload divider so next toggle happens after BAUD cycles.

Power-Saving: Optionally, clock gating can be used when SPI is idle (disable the divider and stop toggling SCLK to save power).

Design Decision (Clock Divider):

Beginner: Use a binary counter that increments each pclk_i. When it reaches the BAUD value, flip a clk_out bit and reset

counter to 0.

Intermediate: Ensure glitch-free switching: update BAUD register synchronously (e.g. only take effect between transfers). Avoid glitch on CPOL change by only changing CPOL when SPI idle.

Senior: To allow maximum throughput, ensure divider logic can toggle SCLK on one cycle without extra latency. Also consider clock gating (e.g. `spi_clk_o = core_clk & enable`) or using clock enables to save power when idle.

4.3 TX FIFO

A FIFO queue buffers bytes/words to transmit. Typical parameters: default depth 16 entries, data width = DATA_WIDTH. It can be instantiated with a dual-port RAM or registers. Key points:

Push Interface (Write): CPU writes to

TXDATA, which pushes into FIFO (if not full). Status bits (e.g. full flag) should be provided or TXDATA write could block/ignore.

Pop Interface (Read): SPI engine reads (pops) from FIFO one frame at a time when it starts a transfer. The engine must detect TX underflow: if FIFO empties before the programmed length of transfer, set an underrun error.

Levels/Flags: Provide signals or status bits for FIFO almost-full, almost-empty, etc., as needed. E.g., an interrupt on TX Empty (FIFO goes empty mid-transfer).

Overflow/Underrun: If CPU writes when FIFO is full, that's an overflow (should be flagged). If SPI engine reads when FIFO empty during shifting, that's an underrun (flag as error).

| **Design Decision (TX FIFO depth):** | | - *Beginner:*
A depth of 16-32 is common for SPI; smaller

FIFOs save area. Use 16 as a default. | | -

Intermediate: FIFO depth affects burst capability.

For large transfers (e.g. DMA bursting 64 bytes), a deeper FIFO (e.g. 64) avoids needing to stall. | |

- *Senior:* Parameterize FIFO depth. DV should test extremes: write to full FIFO (overflow), read from empty FIFO (underrun). Check flags (TXFIFOFULL, TXFIFOEMPTY) produce correct IRQs. Ensure handshake signals (write enables) sync properly across domains. |

According to a Microchip SPI controller example, two 4×32 FIFOs are used. In our design, depth and width are parameters (FIFO_DEPTH, DATA_WIDTH). Implement FIFO protection: on overflow/underflow, capture error bits.

4.4 RX FIFO

Likewise, the RX FIFO buffers received frames from the slave. Key points:

Push (SPI → FIFO): When the SPI engine completes a frame reception, it pushes the word into RX FIFO (if not full). If RX FIFO is full (and another frame arrives), set an overflow error.

Pop (Read): CPU reads from RXDATA to pop FIFO. If FIFO empty and CPU reads, return default (e.g., 0) or stall, but should not cause design error.

Interrupt: A common interrupt is on RX FIFO Full or Data Ready. DV should verify that each pushed value is eventually read correctly. Check the order (should be first-in-first-out).

FIFO Operation (Summary):

Both FIFOs operate similarly and may be implemented with synchronous dual-clock FIFOs (core clock vs SPI clock). Ensure that FIFO write/read enable signals cross domains safely (or

use dual-clock FIFO macros).

4.5 SPI Protocol Engine (Master/Slave FSM)

The heart of the design is the SPI FSM which performs bit-shifting. It contains two sub-FSMs or statesets (depending on mode):

Master FSM: Controls transfers when the core is master. Typical states: IDLE → LOAD (load shift register from TX FIFO) → ASSERT_CS → SHIFT (toggle clocks and shift bits) → COMPLETE → DEASSERT_CS → IDLE.

Slave FSM: Waits for SS_n, then on each SCLK edge shifts bits, and after SS_n deassertion, goes back to IDLE. (If multi-master bus, SS_n might float high normally.)

Bit Counter: Keeps track of bits remaining in current word. When it reaches zero, stop shifting (go to COMPLETE).

Operations: On each active SCLK edge (rising or falling, depending on CPHA), sample MOSI/MISO and shift new bits.

SS_n asserted/Start

TX data available

begin transfer

(start clocking)

last bit shifted

finish transfer

IDLE

LOAD

ASSERT_CS

SHIFT

COMPLETE

DEASSERT_CS

Figure 2. Master FSM for SPI transfers.

CPOL/CPHA Timing: The FSM uses CPOL (clock idle level) and CPHA (which edge to sample) to decide when to toggle SCLK and when to

sample/shift. For example, in [Mode 0 \(CPOL=0, CPHA=0\)](#), data is sampled on the first rising edge (Figure 3). Mode definitions are given later (see Section 5). The FSM should implement the 4-mode truth table exactly.

Clock and Data Timing: On each relevant SCLK edge: the master drives the next MOSI bit (from MSB or LSB depending on setting) and samples the slave's MISO into a shift register. The slave simultaneously drives its MISO from its shift register. After the last bit, the engine deasserts SCLK (ensuring CPOL idle) and optionally deasserts SS_n.

Design Note: In master mode, the SPI engine controls SCLK. In slave mode, treat SCLK as input and generate an internal 'sample clock' by detecting edges (with a small delay counter if needed).

State Machine (Example Master): The IDLE/LOAD/ASSERT/SHIFT/COMPLETE sequence can

be drawn as above. On entry to LOAD, the engine reads one word from TX FIFO into a shift register. In SHIFT state, a bit counter (e.g. 0..DATA_WIDTH-1) counts down on each sampled clock. When count=0, it moves to COMPLETE.

| Design Decision (State Machine): || - *Beginner*: Use a register (e.g. state) with defined states as in Figure 2. A case statement in Verilog can implement transitions. || - *Intermediate*: Ensure the state transitions occur synchronized to pclk_i. Use one-hot encoding or Gray code for state registers to ease synthesis. Watch the timing of SS_n: it must be asserted *before* the first SCLK and deasserted *after* the last SCLK, obeying min assertion times from datasheets. || - *Senior*: Merge states where possible (e.g. ASSERT_CS and SHIFT can share logic) to optimize timing. Consider an output-enable for loopback mode (drive MOSI and MISO together

for self-test). Ensure the FSM meets timing at target clock (synthesis-friendly structure). |

4.6 Interrupt Controller

This logic collects various status flags and generates `spi_irq_o` when enabled events occur. Typical interrupts include TX FIFO Empty, RX FIFO Full, transfer complete, mode fault detected, frame end, and errors. For example, as shown in a reference IP: Interrupt Generation: TX empty, RX full, transfer complete, error.

A simple model:

Each event has an enable bit in `INT_EN` and a sticky status bit in `INT_ST`.

On event (e.g. TX FIFO is now empty), set corresponding bit in `INT_ST` and if the bit is enabled, assert `spi_irq_o`.

Reading `INT_ST` should clear the bits (or use separate write-to-clear mechanism).

STAT register may mirror these flags as well.

DV Perspective: Verify each interrupt line: e.g., if TX FIFO becomes empty during transmission and the enable bit was 1, then spi_irq_o should pulse (level) and INT_ST should capture that event. Common bug: missing clear of IRQ or spurious IRQ when writing status register.

4.7 DMA Support (Optional)

If implementing DMA, add handshake signals (e.g. tx_dma_req, rx_dma_req). The controller could raise a DMA request when TX FIFO empties or RX FIFO fills. DMA engine logic is system-dependent, so we just note:

Recommend: Include a dma_req_o output or use spi_irq_o as DMA trigger. Provide an enable bit for DMA mode.

DV Note: Verify that DMA requests coincide with threshold conditions and that normal

APB access still functions.

5. SPI Modes and Timing

SPI supports four modes determined by clock [polarity \(CPOL\)](#) and [phase \(CPHA\)](#):

Mode	CPOL	CPHA	Clock Idle	Data Sample Edge
0	0	0	Low	Rising edge (first edge)
1	0	1	Low	Falling edge

				(second edge)
2	1	0	High	Falling edge (first edge)
3	1	1	High	Rising edge (second edge)

Table 3. [SPI Modes \(CPOL/CPHA\) and timing](#) (from ref. IP).

These modes correspond to whether data is sampled on the first or second clock transition relative to the SS assertion. For example, in [Mode 0 \(CPOL=0, CPHA=0\)](#), SCLK idles low, and

data is captured on the rising edge (Figure 3).
Mode 1 shifts/stores opposite edges, etc..

csharpCopy



Figure 3. SPI timing diagrams (simplified): example showing MOSI (Master → Slave) and SCLK for Modes 0–3. Arrows indicate the sampling edge (when MISO is captured, and the current MOSI bit must be stable).

Interpretation: The SCLK line's idle level (when no transfer is active) is CPOL. When a transfer starts (SS_n asserted), the first SCLK edge occurs after a short setup time, as defined by CPHA. The slave reads MOSI at the indicated "sample" edge; on the opposite edge data is shifted out. The Analog Devices article explains: "CPOL sets the idle state of the clock. CPHA selects which edge is used for sample/shift." DV engineers must test all four modes to ensure correct edges are used.

Timing Parameters: In implementing, ensure all timing (setup/hold of MOSI relative to SCLK edges, minimum pulse widths, SS_n timing) meet target specs. The APB CLKDIV (BAUD)

determines SCLK frequency; system must guarantee $pclk_i/4 \geq SCLK$ for stable sampling.

Bit/Byte Transfer Example: For clarity, a byte transfer (8 bits) in Mode 0: the master asserts SS_n, then toggles SCLK 8 times. On each rising edge, MOSI's current bit is sampled by slave, and simultaneously the slave changes MISO for the next bit. After 8 clocks, master deasserts SS_n. This results in 8 bits full-duplex exchange.

5.1 Error Conditions (Mode Fault)

Mode Fault (MODF): In multi-master setups, if two masters assert SS_n at the same time, a mode fault should be detected. The hardware monitors SS_n: if the master sees SS_n asserted low when it believed none was (or if in slave mode it detects another master), it flags a mode fault. The reaction may be to abort the transaction and raise an error flag.

Beginner: Mode fault means another master is active.

Intermediate: In master mode, implement logic: if SS_n is an input (in multi-master case), and it goes low unexpectedly, stop driving SCLK and set MODE_FAULT flag.

Senior: Formalize arbitration policy (priority scheme) if needed. At minimum, SRL/MODF status bit and interrupt should be provided. DV must test by simulating two masters on one bus (or manually toggling SS) to ensure the flag triggers.

[5.2](#) Error Conditions (FIFO Under/Overflow)

As noted in the SmartFusion manual:

TX Underrun: If TX FIFO is empty before the SPI frame is fully shifted, set TX_UNDERRUN (similar to “TX underrun” in Microchip doc).

RX Overflow: If RX FIFO is full when a new frame arrives, set `RX_OVERFLOW`.

Error bits should be sticky (in status register) until cleared by software.

DV Check: Force an underrun (e.g. start a long transfer without loading enough TX data) and confirm `STAT.TX_UNDERRUN` and interrupt if enabled. Likewise for overflow.

6. Register Map

A summary register map (with recommended defaults in parentheses) is provided below.

Address offsets are examples (assuming APB base address is offset 0x0):

Offset	Regist	Bits/	Access	Descri
--------	--------	-------	--------	--------

	er	Fields		ption
0x00	CTRL	[0]=EN, [1]=MA STER, [2]=ST ART, [3]=LS B_FIRS T, [4]=CP OL, [5]=CP HA, [7:6]=Fr ameLe n (2 bits)	R/W	Control : enable core, select master /slave, LSB/ MSB first, start transfe r, CPOL/ CPHA, frame length (in

				bytes)
0x04	STAT	[0]=BUSY, [4:1]=TXFILL, [8:5]=RXFILL, [9]=MODE, [10]=UNDERRUN, [11]=OVERFLOW	R	Status: busy flag, FIFO levels, mode fault, underrun/ overflow
0x08	TXDATA	[7:0]=DataOut	W	Write data to TX FIFO

				(width = DATA_WIDTH / 8 bytes)
0x0C	RXDATA	[7:0]=DataIn	R	Read data from RX FIFO
0x10	BAUD	[15:0]=Divider	R/W	Clock divisor (divides core clock: actual SCLK freq =

				pclk_i/ (divider +1)/2)
0x14	FIFO	[0]=FIFO_EN, [1]=TX_FIFO_HALT, [2]=RX_FIFO_HALT	R/W	FIFO control : enable FIFOs, pause FIFO pointers (for flush).
0x18	INT_EN	[0]=TX_EMPTY_IE, [1]=RX_FULL_IE,	R/W	Interrupt enable s for TX empty,

		[2]=MO DF_IE, [3]=RX _OVF_I E, [4]=TX _UDR_I E		RX full, mode fault, RX overflow, TX underrun
0x1C	INT_ST	[0]=TX _EMPTY Y, [1]=RX _FULL, [2]=MO DF, [3]=RX _OVF, [4]=TX	R/W1C	Interrupt status (write 1 to clear)

		_UDR		
0x20	MODE	[0]=MASTER (alias), [2:1]=CPOL/ CPHA (alias), [7:3]=SSI (multi-slave select)	R/W	SPI mode and slave select (alternate way to set CPOL/ CPHA)

Table 3. Example SPI controller register map (offsets and fields).

(Note: Actual implementation may reorder or combine fields. All unspecified bits should be reserved and read as 0.)

DV engineers will use these registers to drive transfers in tests. For instance, a typical sequence to send 3 bytes:

pgsqlCopy



```
WRITE CTRL = 0x?? (set MASTER=1,  
CPOL/CPHA, LSB/MSB, frameLen=3)  
WRITE TXDATA (byte0) → TX FIFO  
WRITE TXDATA (byte1)  
WRITE TXDATA (byte2)  
WRITE CTRL.START (set start bit to 1)  
// SPI engine runs, STAT.BUSY goes high  
  
WAIT until STAT.BUSY clears; optionally  
read RXDATA thrice for any MISO bytes.
```

Testing should cover all fields (e.g., toggling LSB_FIRST, enabling/disabling FIFO, halting FIFO pointers, etc.).

7. Timing Diagrams and Data Flow

7.1 Example Transfers

Byte Transfer: In master mode, a byte transfer (8-bit, mode 0) proceeds as in Figure 3: SS_n asserted, then 8 clock pulses on SCLK; each rising edge sends one MOSI bit and samples one MISO bit; then SS_n deasserted.

Multi-byte Burst: For a longer transfer of N bytes, the SPI engine continues shifting back-to-back as long as SS_n is held low and words are available. On each word boundary, it automatically reloads the shift register from TX FIFO and pushes to RX FIFO. For example, with frameLen=4, 32 clocks occur.

Three-wire or 3-phase: Standard SPI uses four wires. Some variants (not covered here) share

MOSI/MISO or use 3-wire mode (with data in/out multiplexed). These are beyond this spec's scope.

7.2 Setup/Hold Requirements

Within the core design, assume the following timing:

CPU to TXDATA: Setup/hold not critical if TXDATA write is synchronous to `pclk_i`.

SS_n Assertion: When CTRL.START is written, assert SS_n on next `pclk` rise (or next SPI clock edge). Provide a few `pclk_i` cycles of setup before the first SCLK to meet slave setup time.

Setup/Hold of MOSI: The master must present the MOSI bit stable at least setup time before the sampled clock edge, and hold it after the edge. The SPI engine should align MOSI shift with SCLK edges so that

timing meets slave specs.

Sampling MISO: At each sample edge, the SPI engine latches MISO. This is in SPI clock domain. The design should ensure MISO is stable on that edge (slave design's responsibility). If the core is in slave mode, it must sample on the correct edge and meet setup/hold on incoming SCLK.

Timing diagrams (in realistic scenarios) depend on physical delays. Designers should add small guards and test for metastability by brute force or formal checking (see assertions below).

8. Clocking, Reset, and Power

Clock Domains: As noted, assume one main clock (pclk_i). The SPI clock (SCLK) is

derived by a divider. All internal logic (FSM, FIFOs, ctrl) runs on pclk_i. If implementing slave mode with external SCLK, those signals are sampled into the pclk_i domain.

Reset: Provide an asynchronous or synchronous reset (presetn_i). On reset:

- All registers (CTRL, BAUD, MODE, etc.) should go to default values (e.g., disable core, CPOL=0, CPHA=0).

- FIFOs are cleared (empty).

- Interrupt flags and spi_irq_o cleared.

- The SPI lines (spi_clk, spi_mosi, spi_csn, spi_irq) should go to known idle states (clk=CPOL, csn=1, mosi=0).

DV Note: Verify reset does clear all state. A common bug is not resetting handshake flops, leading to Xs.

Power Management (Optional): We may include a low-power mode: when CTRL.EN is 0, clock gating turns off internal toggling

(except pclk_i may run). When idle, gates internal registers or stops the clock divider. Provide a bit in CTRL (or separate register) to enter power-down.

DV Note: Test a sequence of enabling/disabling during operation, ensuring no illegal behavior (e.g. system recovers correctly).

9. Verification Hooks and Testpoints

Monitors and Scoreboard: Design the IP to allow easy monitoring:

Provide a "chip select" handshake (SS_n) event and clock transitions for a SPI Monitor in DV to latch all bits.

The APB interface can be observed by a CSR

monitor.

Optionally include testpoints (untouched internal nets) for monitoring (e.g. expose an internal “shift_done” signal via a debug bus if security/performance permit).

For example, the OpenTitan DV plan uses a scoreboard that monitors TL (host) and SPI transactions end-to-end. Our design should allow such a scoreboard: it can compare the sequence of bytes written to TXDATA and read from RXDATA to expected patterns.

Structural Coverage Points: Internally, ensure:

The FSM visits all states for each mode.

All register fields take on both 0 and 1 in tests.

FIFOs empty/full states are reachable.

Interrupt paths are exercised.

DV should write stimuli to cover those.

10. SVA Assertions

Include strong assertions (SystemVerilog Assertions) to catch protocol violations.

Examples:

Property	Description
<i>p_ss_active</i> : Always: during any data transfer cycle, SS_n must be asserted.	Ensures SS_n remains low while shifting data. ``
property p_ss_active;	
@(posedge pclk_i) (CTRL.START ##1 ! spi_csn_o);	

endproperty

ensuresCopy

Ensures



| **p_clk_idle**: If no transfer, SCLK must equal CPOL. | Guarantees idle clock level.
```

```
property p_clk_idle;
```

```
  @(posedge pclk_i) (!CTRL.START |  
  STAT.BUSY) |-> (spi_clk_o == CPOL)
```

```
endproperty
```

```
``` |
```

| *\*p\_mosi\_stable\**: MOSI must be stable around the sampling edge. | Prevent data glitch at sample. <br>```

```
property p_mosi_stable;
```

```
 @(posedge spi_clk) disable iff
 (spi_csn_o) $stable(spi_mosi_o);
```

```
endproperty
```

```
``` |
```

| **p_data_shift**: After 8 clocks (DATA_WIDTH), BUSY must drop. |

